

Kubernetes Node NotReady Recovery Runbook

Document ID: RB-K8S-003 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	Severity	P1 - Critical
Domain	Container Orchestration	Est. Duration	30-60 minutes
Tags	kubernetes, node, notready, kubelet, drain, cordon		

Overview

A NotReady node means the kubelet on that node has stopped reporting to the API server. All pods on the node are evicted or stuck in Terminating state. This affects all workloads scheduled on the node and may cause cascading failures if multiple nodes are affected.

Prerequisites

- SSH access to the affected node
- kubectl with cluster-admin permissions
- Node group auto-scaling access (AWS/GCP/Azure)
- Access to node system logs (journald or /var/log)

Response Steps

Step 1: Identify the NotReady Node

List all nodes and identify which ones are in NotReady state and how long they have been down.

```
kubectl get nodes -o wide
kubectl get nodes --no-headers | grep -v Ready
```

NOTE: Note the AGE column — a node down >5 minutes has likely evicted its pods already.

Step 2: Describe Node for Root Cause

Inspect node conditions, resource pressure, and recent events to determine whether it is a disk, memory, or network issue.

```
kubectl describe node
kubectl get events --field-selector involvedObject.name= -A
```

Step 3: SSH to Node and Check Kubelet

SSH to the node and check kubelet service status. Most NotReady conditions are caused by a stopped or crashed kubelet.

```
ssh
systemctl status kubelet
journalctl -u kubelet -n 50 --no-pager
```

Step 4: Restart Kubelet if Stopped

If kubelet is stopped or failed, restart it and check it returns to active state.

```
systemctl restart kubelet
systemctl status kubelet
sleep 30 && kubectl get node
```

NOTE: Allow 30 seconds for the node to re-register with the API server.

Step 5: Check Disk and Memory Pressure

If kubelet is running but node is still NotReady, check for DiskPressure or MemoryPressure conditions.

```
df -h
free -m
du -sh /var/lib/docker/* | sort -rh | head -10
```

Step 6: Cordon and Drain if Unrecoverable

If the node cannot be recovered quickly, cordon it to stop new scheduling and drain all pods to healthy nodes.

```
kubectl cordon
kubectl drain --ignore-daemonsets --delete-emptydir-data --force
kubectl get pods -A -o wide | grep
```

Step 7: Replace Node via Node Group

Terminate the unrecoverable node via the cloud provider. The auto-scaler will provision a replacement.

```
# AWS: aws ec2 terminate-instances --instance-ids
# GCP: gcloud compute instances delete --zone=
kubectl get nodes -w
```

NOTE: SUCCESS: New node appears in Ready state within 3-5 minutes.

Rollback Steps

Rollback Step 1: Uncordon Node if Recovered

If the node recovered after kubelet restart, uncordon it to resume scheduling.

```
kubectl uncordon
kubectl get node
```

Rollback Step 2: Re-schedule Critical Pods

If critical pods did not reschedule automatically, delete them to trigger rescheduling.

```
kubectl get pods -A -o wide | grep Pending
kubectl delete pod -n
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Identify Node	—	—
Step 2: Describe Node	Step 1	—
Step 3: SSH + Check Kubelet	Step 1	Step 2
Step 4: Restart Kubelet	Step 3	—
Step 5: Check Disk/Memory	Step 3	Step 4
Step 6: Cordon and Drain	Step 4, Step 5	—
Step 7: Replace Node	Step 6	—